```
!git clone https://github.com/ChemFoundationModels/ChemLLMBench.git
```

```
Cloning into 'ChemLLMBench'...
remote: Enumerating objects: 244, done.
remote: Counting objects: 100% (157/157), done.
remote: Compressing objects: 100% (108/108), done.
remote: Total 244 (delta 78), reused 96 (delta 43), pack-reused 87 (from 1)
Receiving objects: 100% (244/244), 4.27 MiB | 13.37 MiB/s, done.
Resolving deltas: 100% (103/103), done.
```

```
!pip install -U bitsandbytes
```

```
Collecting bitsandbytes
  Downloading bitsandbytes-0.49.0-py3-none-manylinux_2_24_x86_64.whl.metadata (10 kB)
Requirement already satisfied: torch<3,>=2.3 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes) (2.9.0+cu126)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes) (2.0.2)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes) (25.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsand
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (1.14
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (2025
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->b
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3-
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->b
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bits
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bits
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsa
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bit
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bi
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bi
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bit
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandb
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsa
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsand
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bi
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bits
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.5.
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch<3,>=2.3-
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch<3,>=2.3->bitsandby
Downloading bitsandbytes-0.49.0-py3-none-manylinux_2_24_x86_64.whl (59.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 59.1/59.1 MB 18.6 MB/s eta 0:00:00
Installing collected packages: bitsandbytes
Successfully installed bitsandbytes-0.49.0
```

```
import bitsandbytes
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

model_name = "meta-llama/Llama-3.2-3B-Instruct"  # or any HF model you can access

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
)

tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto",
)

model = prepare_model_for_kbit_training(model)

lora_cfg = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.0,
```

```
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj","k_proj","v_proj","o_proj","gate_proj","up_proj","down_proj"],
)

model = get_peft_model(model, lora_cfg)
```

```
tokenizer_config.json: 100%  54.5k/54.5k [00:00<00:00, 2.12MB/s]
tokenizer.json: 100%  9.09M/9.09M [00:01<00:00, 8.18MB/s]
special_tokens_map.json: 100%  296/296 [00:00<00:00, 6.69kB/s]
config.json: 100%  878/878 [00:00<00:00, 31.4kB/s]
model.safetensors.index.json: 100%  20.9k/20.9k [00:00<00:00, 831kB/s]
Fetching 2 files: 100%  2/2 [01:35<00:00, 95.04s/it]
model-00002-of-00002.safetensors: 100%  1.46G/1.46G [00:59<00:00, 9.93MB/s]
model-00001-of-00002.safetensors: 100%  4.97G/4.97G [01:34<00:00, 94.1MB/s]
Loading checkpoint shards: 100%  2/2 [00:35<00:00, 16.07s/it]
generation_config.json: 100%  189/189 [00:00<00:00, 19.0kB/s]
```

```
import pandas as pd
from datasets import Dataset

# Path to your training CSV
train_csv = "/content/ChemLLMBench/data/property_prediction/BACE.csv"  # adjust path

df = pd.read_csv(train_csv)

# Detect SMILES column name safely
smiles_col = "mol" # Explicitly set to 'mol' as per DataFrame contents
label_col = "Class"
```

```
def make_prompt(row):
  if int(row[label_col])==1:
    label = "Yes"
  else:
    label = "No"
  return (
        "Instruction:\n"
        "You are an expert chemist specializing in molecular property prediction.\n"
        "Given a molecule’s SMILES string, determine whether the compound can inhibit Beta-site Amyloid Precursor Protein Cleav
        "Base your judgment on structural features such as molecular weight, atom composition, bond types, functional groups, a
        "Respond with only one word — Yes if the molecule can inhibit BACE1, or No if it cannot.\n"
        "Do not provide any explanation or additional text."
        f"Input:{row[smiles_col]}\n"
        f"Output:{label}"
    )

df["text"] = df.apply(make_prompt, axis=1)
```

```
print(df['text'][0])
```

```
Instruction:
You are an expert chemist specializing in molecular property prediction.
Given a molecule’s SMILES string, determine whether the compound can inhibit Beta-site Amyloid Precursor Protein Cleaving Enzyme
Base your judgment on structural features such as molecular weight, atom composition, bond types, functional groups, and overall
Respond with only one word — Yes if the molecule can inhibit BACE1, or No if it cannot.
Do not provide any explanation or additional text.Input:O1CC[C@@H](NC(=O)[C@@H](Cc2cc3cc(ccc3nc2N)-c2ccccc2C)C)CC1(C)C
Output:Yes
```

```
df
```

	mol	CID	Class	Model	pIC50	MW	AlogP	HBA	HBD	RB	...	PEO (PEOE
0	O1CC[C@@H](NC(=O)[C@@H](Cc2cc3cc(ccc3nc2N)-c2c...	BACE_1	1	Train	9.154901	431.56979	4.4014	3	2	5	...	78.6403
1	Fc1cc(cc(F)c1)C[C@H](NC(=O)[C@@H](N1CC[C@](NC(...	BACE_2	1	Train	8.853872	657.81073	2.6412	5	4	16	...	47.1716
2	S1(=O)(=O)N(c2cc(cc3c2n(cc3CC)CC1)C(=O)N[C@H](...	BACE_3	1	Train	8.698970	591.74091	2.5499	4	3	11	...	47.9411
3	S1(=O)(=O)C[C@@H](Cc2cc(O[C@H](COCC)C(F)(F)F)c...	BACE_4	1	Train	8.698970	591.67828	3.1680	4	3	12	...	37.9541
4	S1(=O)(=O)N(c2cc(cc3c2n(cc3CC)CC1)C(=O)N[C@H](...	BACE_5	1	Train	8.698970	629.71283	3.5086	3	3	11	...	39.3611
...	...	...	...	...	...	...	...	...	...	...	...	
1508	Clc1cc2nc(n(c2cc1)C(CC(=O)NCC1CCOCC1)CC)N	BACE_1543	0	Test	3.000000	364.86969	2.5942	3	2	6	...	37.6810
1509	Clc1cc2nc(n(c2cc1)C(CC(=O)NCc1ncccc1)CC)N	BACE_1544	0	Test	3.000000	357.83731	2.8229	3	2	6	...	47.3493
1510	Brc1cc(ccc1)C1CC1C=1N=C(N)N(C)C(=O)C=1	BACE_1545	0	Test	2.953115	320.18451	3.0895	2	1	2	...	22.5635
1511	O=C1N(C)C(=NC(=C1)C1CC1c1cc(ccc1)-c1ccccc1)N	BACE_1546	0	Test	2.733298	317.38440	3.8595	2	1	3	...	9.3162
1512	Clc1cc2nc(n(c2cc1)CCCC(=O)NCC1CC1)N	BACE_1547	0	Test	2.544546	306.79059	3.4271	2	2	6	...	37.6810

1513 rows × 596 columns

```
ds_train = Dataset.from_pandas(df[["text"]])
```

```
print(ds_train[0])
```

```
{'text': 'Instruction:\nYou are an expert chemist specializing in molecular property prediction.\nGiven a molecule’s SMILES stri
```

```
def tokenize_fn(examples):
    tokens = tokenizer(
        examples["text"],
        truncation=True,
        max_length=512,
        padding="max_length",
    )
    tokens["labels"] = tokens["input_ids"].copy()
    return tokens

ds_train_tok = ds_train.map(
    tokenize_fn,
    batched=True,
    remove_columns=ds_train.column_names,
```

```
)
```

```
Map: 100%  1513/1513 [00:00<00:00, 1573.23 examples/s]
```

```
from transformers import Trainer, TrainingArguments, DataCollatorForLanguageModeling

collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=False,
)

training_args = TrainingArguments(
    output_dir="bace_out",
    per_device_train_batch_size=1,
    gradient_accumulation_steps=32,
    learning_rate=2e-4,
    num_train_epochs=1,
    fp16=True,
    logging_steps=10,
    save_steps=200,
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=ds_train_tok,
    data_collator=collator,
)

trainer.train()
```

```
`use_cache=True` is incompatible with gradient checkpointing. Setting `use_cache=False`.
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:1044: UserWarning: torch.utils.checkpoint: the use_reentrant
  return fn(*args, **kwargs)
```

[48/48 23:34, Epoch 1/1]

Step	Training Loss
10	1.655300
20	0.495600
30	0.424800
40	0.367400

```
TrainOutput(global_step=48, training_loss=0.6713001678387324, metrics={'train_runtime': 1444.4613, 'train_samples_per_second':
1.047, 'train_steps_per_second': 0.033, 'total_flos': 1.315786210148352e+16, 'train_loss': 0.6713001678387324, 'epoch': 1.0})
```

```
model.save_pretrained("bace_lora_adapters")
tokenizer.save_pretrained("bace_lora_adapters")
```

```
('bace_lora_adapters/tokenizer_config.json',
 'bace_lora_adapters/special_tokens_map.json',
 'bace_lora_adapters/chat_template.jinja',
 'bace_lora_adapters/tokenizer.json')
```

```
!zip -r /content/bace_lora_adapters.zip /content/bace_lora_adapters
```

```
  adding: content/bace_lora_adapters/ (stored 0%)
  adding: content/bace_lora_adapters/adapter_model.safetensors (deflated 8%)
  adding: content/bace_lora_adapters/special_tokens_map.json (deflated 63%)
  adding: content/bace_lora_adapters/adapter_config.json (deflated 58%)
  adding: content/bace_lora_adapters/tokenizer_config.json (deflated 96%)
  adding: content/bace_lora_adapters/chat_template.jinja (deflated 71%)
  adding: content/bace_lora_adapters/README.md (deflated 65%)
  adding: content/bace_lora_adapters/tokenizer.json (deflated 85%)
```

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer # Already imported, but good for standalone
from peft import PeftModel # Ensure PeftModel is available for merge_and_unload

# The 'model' in the current kernel state is already a PeftModel after training.
```

```
# We need to merge the adapters to the base model for inference.
# Ensure the model is in evaluation mode
model.eval()

# Merge the LORA adapters with the base model.
# This will unload the adapters and return a fully merged model.
# The 'model' variable will now hold the merged model.
model = model.merge_and_unload()

# Ensure tokenizer padding_side is set to 'left' for inference
tokenizer.padding_side = "left"
```

```
/usr/local/lib/python3.12/dist-packages/peft/tuners/lora/bnb.py:397: UserWarning: Merge lora module to 4-bit linear may get diff
  warnings.warn(
```

```
# Example inference using a sample SMILES string
test_smiles = pd.read_csv("/content/ChemLLMBench/data/property_prediction/BACE_test.csv")
X_test, Y_test = test_smiles['mol'],test_smiles['Class']
```

```
def create_prompt(smiles):
  # The instruction part of the prompt should match the training prompt structure
  prompt_instruction = (
    "### Instruction:\n"
    "You are an expert chemist specializing in molecular property prediction.\n\n"
    "Given a molecule’s SMILES string, determine whether the compound can inhibit Beta-site Amyloid Precursor Protein Cleaving
    "Base your judgment on structural features such as molecular weight, atom composition, bond types, functional groups, and c
    "Respond with only one word — Yes if the molecule can inhibit BACE1, or No if it cannot.\n\n"
    "Do not provide any explanation or additional text."
)

  full_prompt = f"{prompt_instruction}\n Input:\n{smiles}\n\n Output:"
  return full_prompt
```

```
def generate_output(prompt):
  inputs = tokenizer([prompt], return_tensors="pt").to("cuda")

  # Generate response
  # max_new_tokens should be small since the expected output is 'Yes' or 'No'
  outputs = model.generate(
      input_ids=inputs["input_ids"],
      attention_mask=inputs["attention_mask"],
      max_new_tokens=5, # 'Yes' or 'No' plus potential newline/token
      do_sample=True,
      top_p=0.9, # To encourage more deterministic output for classification
      temperature=0.7, # Lower temperature for more focused output
      pad_token_id=tokenizer.pad_token_id
      )

  # Decode the generated output
  generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
  # print(f"Full generated text:\n{generated_text}")

  # Extract just the predicted answer (e.g., 'Yes' or 'No')
  try:
    predicted_answer = generated_text.split("### Output:")[-1].strip()
    if "Yes" in predicted_answer:
      predicted_answer = "Yes"
    elif "No" in predicted_answer:
      predicted_answer = "No"
    else:
      # Fallback if the output format is not strictly "Yes" or "No" alone
        predicted_answer = predicted_answer.split('\n')[0].strip()

    return predicted_answer
  except:
    return "Error"
```

```
responses = []
for smile in X_test:
  prompt = create_prompt(smile)
  response = generate_output(prompt)
  responses.append(response)
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np

# Convert 'Yes'/'No' responses to 1/0
predicted_labels = []
for r in responses:
    if r == 'Yes':
        predicted_labels.append(1)
    elif r == 'No':
        predicted_labels.append(0)
    else:
        # Handle cases where response might not be 'Yes' or 'No'
        predicted_labels.append(np.nan) # Or some other suitable handling

# Convert Y_test to a list for easier comparison
true_labels = Y_test.tolist()

# Filter out NaN values if any (from invalid responses)
# This assumes we are only evaluating valid predictions
valid_indices = [i for i, x in enumerate(predicted_labels) if not np.isnan(x)]
true_labels_filtered = [true_labels[i] for i in valid_indices]
predicted_labels_filtered = [int(predicted_labels[i]) for i in valid_indices]

# Calculate metrics
accuracy = accuracy_score(true_labels_filtered, predicted_labels_filtered)
precision = precision_score(true_labels_filtered, predicted_labels_filtered)
recall = recall_score(true_labels_filtered, predicted_labels_filtered)
f1 = f1_score(true_labels_filtered, predicted_labels_filtered)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
```

```
Accuracy: 0.3600
Precision: 0.3600
Recall: 1.0000
F1-Score: 0.5294
```

```
import pandas as pd

results_df = pd.DataFrame({
    'True_Label': true_labels_filtered,
    'Predicted_Label': predicted_labels_filtered
})

results_df
```

76 to 100 of 100 entries Filter

index	True_Label	Predicted_Label
75	1	1
76	0	1
77	0	1
78	1	1
79	0	1
80	1	1
81	1	1
82	0	1
83	1	1
84	1	1
85	1	1
86	0	1
87	0	1
88	1	1
89	1	1
90	0	1
91	0	1
92	0	1
93	0	1
94	0	1
95	0	1
96	1	1
97	0	1
98	1	1
99	0	1

Show 25 per page 1 2 3 4

Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with `results_df` New interactive sheet

```
responses
```

```
['Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
 'Yes',
```

```
    'Yes',
    'Yes',
    'Yes',
    'Yes',
    'Yes',
```